

Name of the Student	Guggilla Venkatasai
Reg. No	192425388
GitHub Link	

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

COURSE NAME: Deep Learning

COURSE CODE: MLA04

COURSE OUTCOME COVERED

CO 2:Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches.(BTL-4)

CO Weightage : 15%

Assessment Tool Weightage: 40%

Duration: 90 minutes

Marks: 25

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Name of the student:

Scenario-Based Questions

CO-AT2

QUESTIONS: 05

Total Marks:25

1. Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

Introduction

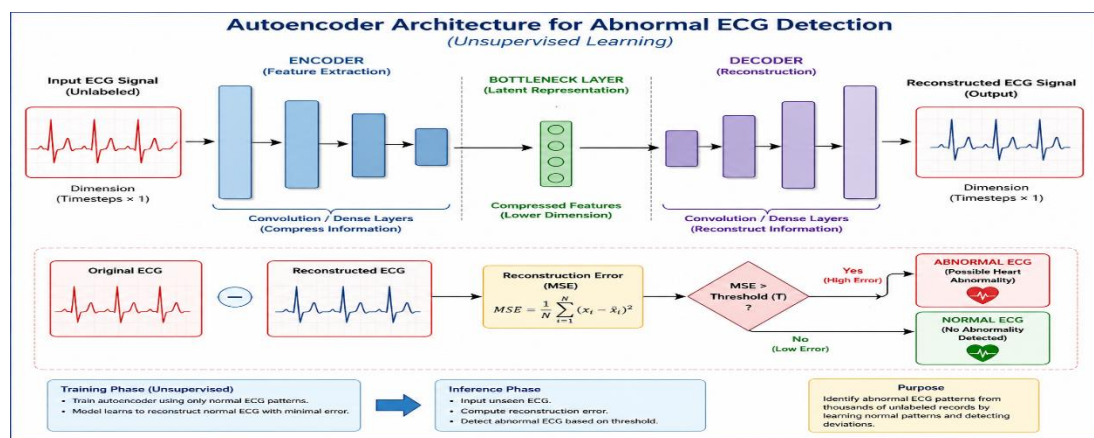
A hospital has collected thousands of ECG (Electrocardiogram) records, but these records are unlabeled, meaning doctors have not identified which ECGs are normal or abnormal. Since there are no labels, supervised learning cannot be used.

The most suitable deep learning technique is an Autoencoder, which is an unsupervised neural network used for representation learning and anomaly detection. An autoencoder learns the normal characteristics of ECG signals automatically. When an abnormal ECG is given, it cannot reconstruct it accurately, producing a high reconstruction error, which helps identify abnormalities.

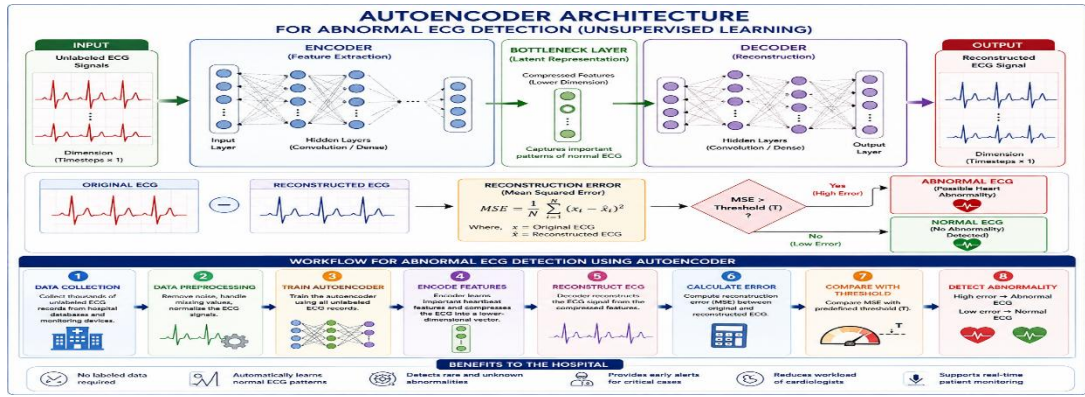
Why Autoencoder?

Autoencoder is recommended because:

- It works with unlabeled data.
- It automatically learns hidden features from ECG signals.
- It reduces high-dimensional ECG data into important features.
- It reconstructs normal ECG signals with very small error.
- Abnormal ECG signals produce large reconstruction errors, making them easy to detect.
- It is widely used in healthcare for anomaly detection, heart disease diagnosis, and patient monitoring.



Workflow



Working Explanation

Step 1: Data Collection

The hospital collects thousands of unlabeled ECG records from patients.

Step 2: Data Preprocessing

Before training the neural network,

- Noise is removed.
- Missing values are handled.
- ECG values are normalized.

This improves learning accuracy.

Step 3: Encoder

The encoder compresses the ECG signal into important features.

Mathematically,

$$h = f(Wx + b)$$

Where

- x = Input ECG signal
- W = Weight matrix
- b = Bias
- f = Activation function (ReLU/Sigmoid)
- h = Hidden feature representation
-

Equation (1)

Step 4: Latent Representation

The bottleneck stores only the most important ECG information.

This process is called **Representation Learning** because the model automatically learns meaningful features without human intervention.

Step 5: Decoder

The decoder reconstructs the original ECG from the compressed representation.

$$\hat{x} = g(W'h + b')$$

Where

- $\hat{x}$ = Reconstructed ECG
- g = Decoder activation function

Equation (2)

Step 6: Calculate Reconstruction Error

The reconstructed ECG is compared with the original ECG using **Mean Squared Error (MSE)**.

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

Where

- x_i = Original ECG
- $\hat{x}_i$ = Reconstructed ECG
- N = Number of ECG samples

Equation (3)

Step 7: Anomaly Detection

The reconstruction error is compared with a threshold.

If

$$MSE > T$$

then

ECG = Abnormal

Otherwise

ECG = Normal

Equation (4)

Algorithm

Input: Thousands of unlabeled ECG records

Output: Normal ECG / Abnormal ECG

Step 1: Collect ECG records.

Step 2: Preprocess ECG signals.

Step 3: Train the Autoencoder using unlabeled data.

Step 4: Encode ECG using Equation (1).

Step 5: Compress into latent representation.

Step 6: Decode using Equation (2).

Step 7: Compute reconstruction error using Equation (3).

Step 8: Compare with threshold using Equation (4).

Step 9: If error is greater than threshold, classify as Abnormal ECG; otherwise classify as Normal ECG.

Advantages

- No labeled data is required.
- Learns important ECG features automatically.
- Detects rare and unknown heart abnormalities.
- Reduces manual analysis by doctors.
- High accuracy for anomaly detection.
- Suitable for large hospital datasets.
- Fast and scalable for real-time monitoring.

Applications

- Heart disease diagnosis
- Arrhythmia detection
- ICU patient monitoring
- Wearable ECG devices
- Remote healthcare systems

2. Auto Encoders, Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

Introduction

A retail company collects a large amount of customer purchase data, such as customer ID, age, gender, income, product categories, purchase frequency, spending amount, payment method, location, and shopping preferences. This high-dimensional data contains redundant and correlated features, making customer segmentation difficult and computationally expensive.

To solve this problem, the company can use an Autoencoder, an unsupervised deep learning model that performs representation learning. An autoencoder compresses the original high-dimensional purchase data into a smaller set of meaningful features while preserving important information. These compressed features are then used for accurate customer segmentation.

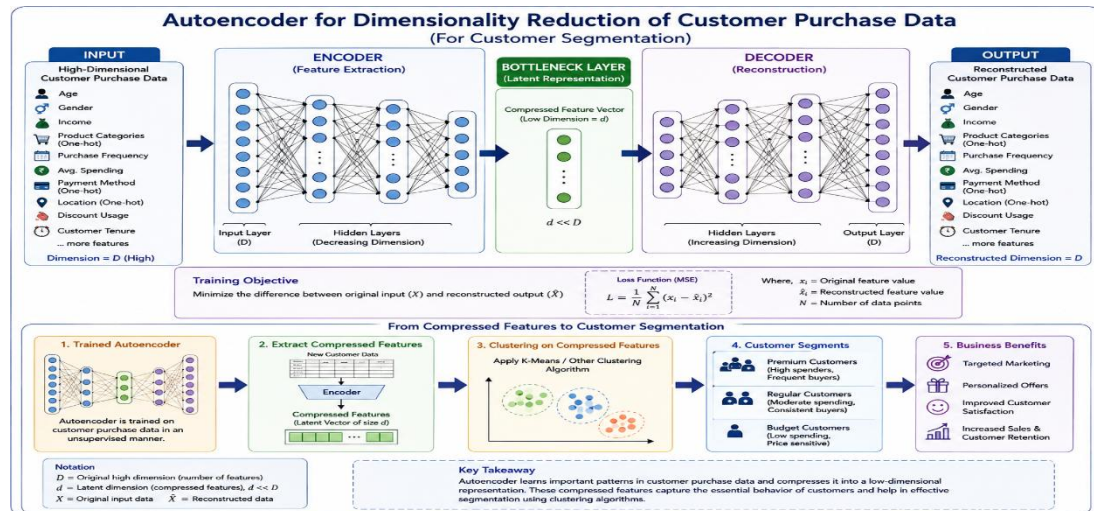
Why Autoencoder?

An Autoencoder is suitable because:

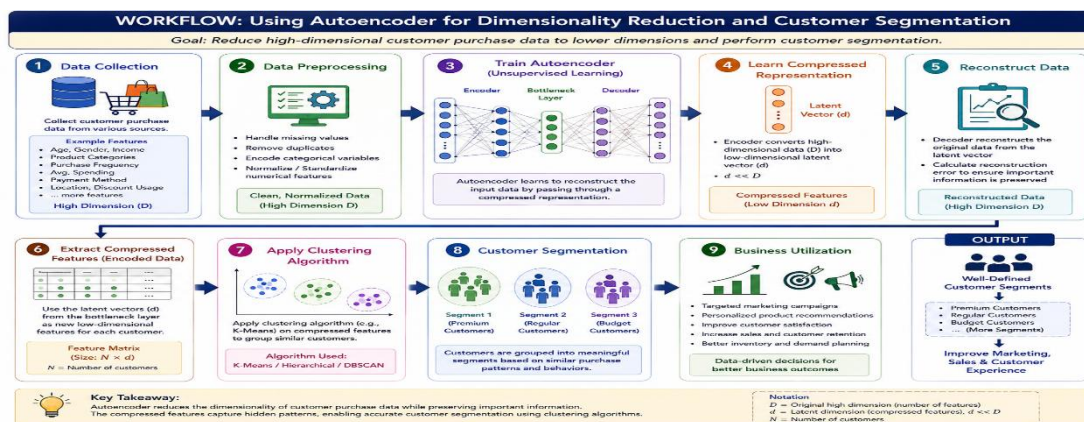
- Customer purchase data is high-dimensional.
- It automatically learns hidden purchasing patterns without requiring labeled data.
- It removes redundant and noisy information.
- It reduces the dimensionality of the dataset.

- The compressed features improve clustering algorithms such as K-Means for customer segmentation.
- It reduces computational cost while maintaining important information.

Architecture Diagram



Workflow



Working Explanation

Step 1: Data Collection

The retail company collects customer purchase information, including:

- Age
- Income
- Products Purchased
- Purchase Frequency
- Total Spending
- Payment Method
- Location
- Shopping Preferences

Step 2: Data Preprocessing

Before training the model:

- Remove duplicate records.
- Handle missing values.
- Normalize feature values.
- Convert categorical data into numerical form if necessary.

Step 3: Encoder

The encoder compresses the customer purchase data into a smaller feature representation.

$$h = f(Wx + b)$$

Where:

x = Customer purchase data

W = Weight matrix

b = Bias

f = Activation function (ReLU/Sigmoid)

h = Latent representation

Equation (1)

Step 4: Representation Learning

The bottleneck layer stores only the most important customer purchasing characteristics.

Examples of learned hidden features:

- High-value customers
- Frequent buyers
- Seasonal shoppers
- Discount-sensitive customers

These features are learned automatically without manual feature engineering.

Step 5: Decoder

The decoder reconstructs the original purchase data from the compressed representation.

$$\hat{x} = g(W'h + b')$$

Where:

$\hat{x}$ = Reconstructed purchase data

g = Decoder activation function

Equation (2)

Step 6: Reconstruction Error

The reconstruction error is measured using Mean Squared Error (MSE).

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

Where:

x_i = Original purchase data

$\hat{x}_i$ = Reconstructed purchase data

N = Number of customer records

Equation (3)

Step 7: Customer Segmentation

The compressed features (latent representation) generated by the encoder are provided as input to a clustering algorithm such as K-Means.

Customers are grouped into segments such as:

- **Premium Customers** – High spending and frequent purchases.
- **Regular Customers** – Moderate spending and consistent purchases.
- **Budget Customers** – Low spending and price-sensitive shoppers.

These segments help the company design targeted marketing campaigns and improve customer satisfaction.

Algorithm

Input: High-dimensional customer purchase dataset

Output: Customer segments

Step 1: Collect customer purchase data.

Step 2: Preprocess the data (cleaning and normalization).

Step 3: Train the Autoencoder using the purchase dataset.

Step 4: Encode the data into compressed features using Equation (1).

Step 5: Decode the compressed features using Equation (2).

Step 6: Calculate reconstruction error using Equation (3).

Step 7: Extract the latent (compressed) features.

Step 8: Apply K-Means clustering on the compressed features.

Step 9: Generate customer segments (Premium, Regular, Budget).

Advantages

- Reduces dimensionality of customer data.
- Removes redundant and noisy features.
- Learns meaningful customer behavior automatically.
- Improves the accuracy of customer segmentation.
- Reduces computation time and storage requirements.
- Supports personalized marketing and product recommendations.

Applications

- Customer segmentation
- Personalized product recommendations
- Targeted marketing campaigns
- Customer behavior analysis
- Sales forecasting
- Loyalty program optimization

3. Width and Depth of Neural Networks, Activation Functions

A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

Introduction

A facial recognition system is designed to identify or verify a person's identity from facial images. However, when the system uses a shallow neural network (having only one or two hidden layers), it cannot learn complex facial features such as eyes, nose, mouth, facial expressions, and lighting variations. As a result, the recognition accuracy becomes poor.

To improve the performance, the network architecture should be modified by increasing the depth of the neural network and selecting appropriate activation functions such as ReLU or Leaky ReLU. A deeper network learns hierarchical facial features, resulting in higher recognition accuracy.

Why Increase Network Depth?

A deep neural network contains multiple hidden layers that learn features at different levels.

- Early layers detect edges and textures.
- Middle layers identify facial parts such as eyes, nose, and lips.
- Deeper layers recognize complete facial structures and identity.
- It captures complex and non-linear relationships in facial images.
- It improves recognition accuracy under different lighting conditions, poses, and facial expressions.

Why Use ReLU / Leaky ReLU Activation Functions?

ReLU (Rectified Linear Unit)

$$ReLU(x) = \max(0, x)$$

Advantages

- Faster training.
- Solves the vanishing gradient problem.

- Computationally efficient.
- Widely used in deep neural networks.

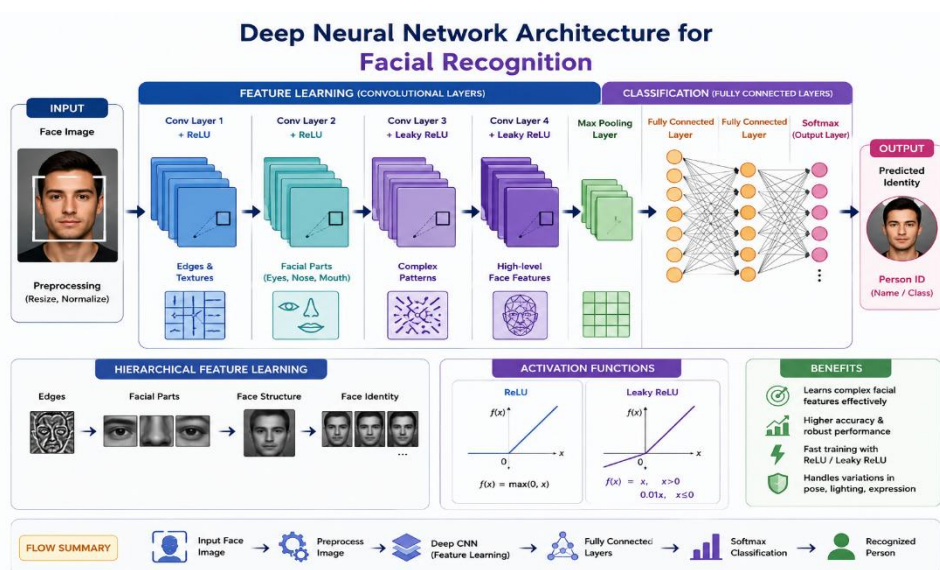
Leaky ReLU

$$\text{LeakyReLU}(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x \leq 0 \end{cases}$$

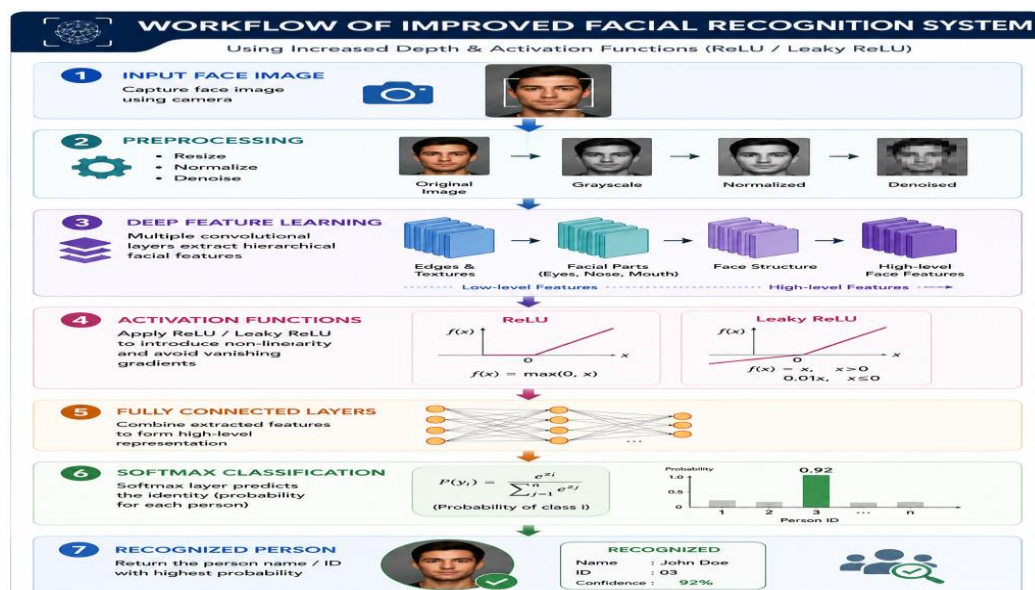
Advantages

- Prevents the "dying ReLU" problem.
- Allows small gradients for negative inputs.
- Improves learning stability.

Architecture



Workflow



Working Explanation

Step 1: Input Image

The system receives a facial image captured by a camera.

Step 2: Image Preprocessing

- Resize image.
- Normalize pixel values.
- Remove noise.
- Improve image quality.

Step 3: Increase Network Depth

Instead of using only one hidden layer, multiple hidden layers are added.

Benefits:

- Learns more complex facial features.
- Improves feature hierarchy.
- Increases recognition accuracy.

Step 4: Feature Extraction

Different layers learn different information.

Layer	Features Learned
Layer 1	Edges and textures
Layer 2	Eyes, nose, mouth
Layer 3	Face shape
Layer 4	Identity features

Step 5: Apply Activation Function

Each hidden layer uses ReLU or Leaky ReLU.

ReLU

$$ReLU(x) = \max(0, x)$$

Equation (1)

Leaky ReLU

$$LeakyReLU(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x \leq 0 \end{cases}$$

Equation (2)

Step 6: Fully Connected Layer

The extracted facial features are combined to create a high-level representation for classification.

Step 7: Softmax Classification

The final layer predicts the identity of the person.

The Softmax function is

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Where:

z_i = Output score for class i

$P(y_i)$ = Probability of class i

Equation (3)

Algorithm

Input: Facial image dataset

Output: Recognized person's identity

Step 1: Collect facial images.

Step 2: Preprocess the images (resize and normalize).

Step 3: Increase the depth of the neural network by adding more hidden/convolutional layers.

Step 4: Apply ReLU in the hidden layers using Equation (1).

Step 5: Use Leaky ReLU in deeper layers to avoid the dying ReLU problem using Equation (2).

Step 6: Extract hierarchical facial features.

Step 7: Pass features to the fully connected layer.

Step 8: Apply Softmax using Equation (3) to classify the face.

Step 9: Display the recognized person's identity.

Advantages

- Learns complex facial features effectively.
- Improves recognition accuracy.
- Handles variations in lighting, pose, and facial expressions.
- Faster convergence using ReLU.
- Prevents dead neurons using Leaky ReLU.
- Suitable for large-scale facial recognition systems.

Applications

- Smartphone face unlock
- Airport biometric verification
- Attendance management systems
- Security surveillance
- Banking identity verification
- Smart access control systems

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

Introduction

A streaming service such as Netflix or Amazon Prime stores a large amount of unlabeled user viewing history, including movies watched, ratings, watch time, genres, and user preferences. Since the data is unlabeled, a Restricted Boltzmann Machine (RBM) is an ideal deep learning model for learning hidden patterns in user behavior.

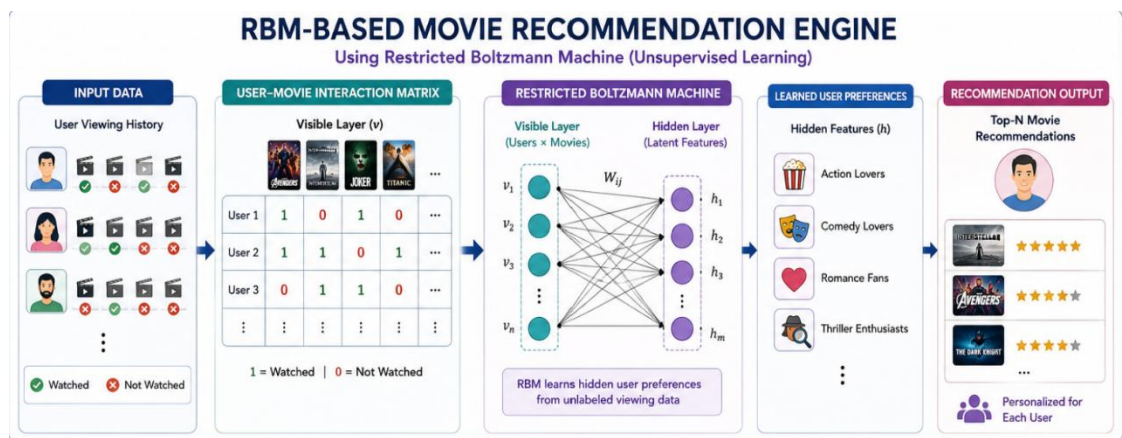
An RBM is an unsupervised neural network consisting of a Visible Layer and a Hidden Layer. It automatically learns relationships between users and movies and predicts which unseen movies a user is likely to enjoy. These predictions are then used to generate personalized movie recommendations.

Why RBM?

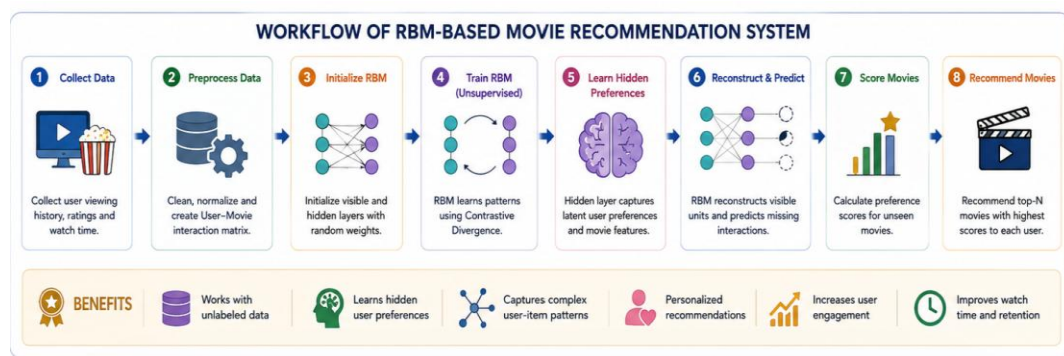
RBM is recommended because:

- Works with **unlabeled user viewing data**.
- Learns hidden preferences of users automatically.
- Identifies similarities between users and movies.
- Predicts missing ratings for unseen movies.
- Generates personalized recommendations.
- Improves user satisfaction and engagement.

Architecture



Workflow



Working Principle

Step 1: Data Collection

The streaming service collects:

- User viewing history
- Movie ratings
- Watch time
- Favorite genres
- Search history

Step 2: Data Preprocessing

- Remove incomplete records.
- Normalize ratings.
- Create a User–Movie Interaction Matrix.

Step 3: Visible Layer

The visible layer represents user interactions with movies.

Example:

User	Movie A	Movie B	Movie C
U1	1	0	1

Here,

1 = Watched

0 = Not Watched

Step 4: Hidden Layer

The hidden layer automatically learns latent features such as:

- Action movie lovers
- Comedy lovers
- Romantic movie fans
- Thriller enthusiasts

These hidden features represent user preferences that are not explicitly stored.

Step 5: Learning User Preferences

The RBM learns the relationship between visible and hidden neurons.

Hidden neuron probability:

$$P(h_j = 1 | v) = \sigma \left(\sum_i W_{ij} v_i + b_j \right)$$

Where

v_i = Visible neuron

W_{ij} = Weight

b_j = Bias

σ = Sigmoid activation

Equation (1)

Step 6: Reconstruct User Preferences

The RBM predicts the probability that a user will like unseen movies.

Visible neuron probability:

$$P(v_i = 1 | h) = \sigma \left(\sum_j W_{ij} h_j + a_i \right)$$

Where

h_j = Hidden neuron

a_i = Visible layer bias

Equation (2)

Step 7: Movie Recommendation

Movies with the highest predicted probabilities are recommended to the user.

Example:

User	Recommended Movies
User A	Avengers, Interstellar
User B	Inception, The Matrix

Algorithm

Input: User viewing history

Output: Personalized movie recommendations

Step 1: Collect user viewing history and ratings.

Step 2: Preprocess the data and create the User–Movie matrix.

Step 3: Train the RBM using unlabeled viewing data.

Step 4: Learn hidden user preferences using Equation (1).

Step 5: Reconstruct user preferences using Equation (2).

Step 6: Predict unseen movie ratings.

Step 7: Rank movies according to predicted scores.

Step 8: Recommend the highest-ranked movies to each user.

Advantages

- Requires no labeled data.
- Learns hidden user interests automatically.
- Generates personalized recommendations.
- Handles large movie datasets efficiently.
- Improves recommendation accuracy.
- Enhances user engagement and watch time.

Applications

- Netflix movie recommendations
- Amazon Prime Video suggestions
- YouTube video recommendations
- Spotify music recommendations
- E-commerce product recommendations

5. Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

Introduction

A manufacturing company continuously collects sensor readings every second from machines during the production process. These sensors monitor parameters such as temperature, pressure, vibration, humidity, and machine speed. Since the data is generated continuously and defects may not always be labeled, a deep learning model using representation learning is an effective solution.

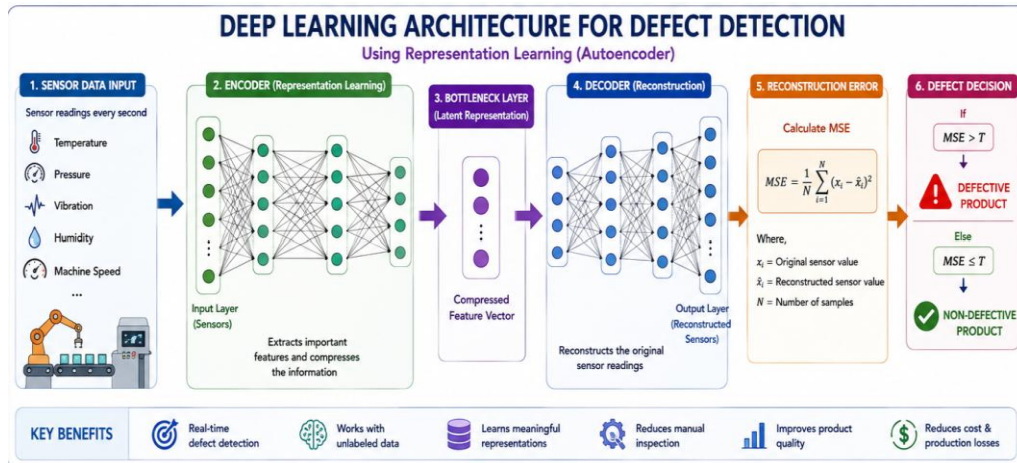
A suitable approach is to use an Autoencoder-based Neural Network, which automatically learns the normal behavior of sensor data. Products with abnormal sensor patterns produce a high reconstruction error and are identified as defective.

Why Autoencoder with Representation Learning?

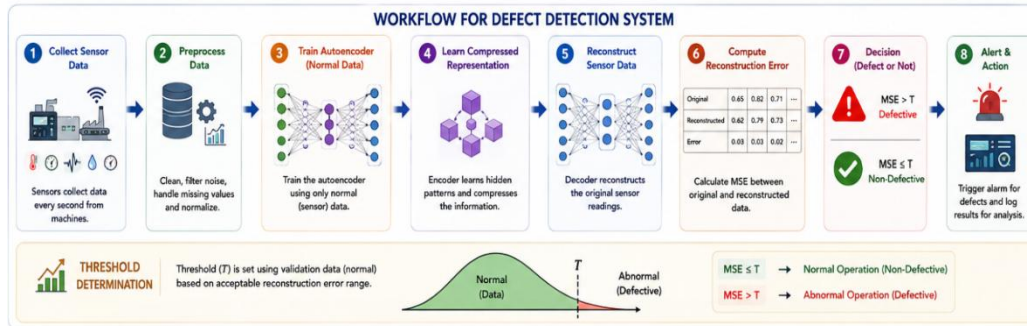
An Autoencoder is recommended because:

- Works effectively with unlabeled sensor data.
- Learns hidden patterns from normal manufacturing conditions.
- Reduces high-dimensional sensor readings into meaningful features.
- Detects abnormal sensor behavior using reconstruction error.
- Enables real-time defect detection on production lines.
- Improves product quality while reducing inspection costs.

Architecture



Workflow



Working Explanation

Step 1: Sensor Data Collection

Sensors continuously collect:

- Temperature
- Pressure
- Vibration
- Humidity
- Machine Speed

every second from the production line.

Step 2: Data Preprocessing

Before training,

- Remove noisy sensor readings.
- Handle missing values.
- Normalize all sensor values.

Step 3: Representation Learning (Encoder)

The encoder extracts important features from the sensor readings and compresses them into a latent representation.

$$h = f(Wx + b)$$

Where:

x = Sensor readings

W = Weight matrix

b = Bias

f = Activation function (ReLU)

h = Compressed feature vector

Equation (1)

Step 4: Latent Representation

The bottleneck layer stores only the most important production characteristics, removing redundant information while preserving defect-related patterns.

Step 5: Decoder

The decoder reconstructs the original sensor readings from the compressed features.

$$\hat{x} = g(W'h + b')$$

Where:

$\hat{x}$ = Reconstructed sensor readings

g = Decoder activation function

Equation (2)

Step 6: Reconstruction Error

The reconstruction error is calculated using Mean Squared Error (MSE).

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

Where:

x_i = Original sensor value

$\hat{x}_i$ = Reconstructed sensor value

N = Number of sensor samples

Equation (3)

Step 7: Defect Detection

If

$$MSE > T$$

then

Product = Defective

Otherwise

Product = Non-Defective

Equation (4)

Algorithm

Input: Sensor readings from manufacturing machines

Output: Defective or Non-Defective product

Step 1: Collect sensor readings every second.

Step 2: Preprocess the sensor data.

Step 3: Train the Autoencoder using normal sensor data.

Step 4: Encode sensor data using **Equation (1)**.

Step 5: Learn compressed representations.

Step 6: Decode using **Equation (2)**.

Step 7: Calculate reconstruction error using **Equation (3)**.

Step 8: Compare the error with the threshold using **Equation (4)**.

Step 9: Classify the product as defective or non-defective.

Advantages

- Real-time defect detection.
- Works without labeled defect data.
- Learns meaningful sensor representations automatically.
- Reduces manual quality inspection.
- Improves manufacturing efficiency.
- Minimizes production losses.

Applications

- Smart manufacturing
- Industrial quality inspection
- Predictive maintenance
- Automotive production
- Electronics manufacturing
- Pharmaceutical production

Conclusion

For this manufacturing company, an Autoencoder-based deep learning model using representation learning is the most suitable solution for detecting defective products from continuous sensor readings. The model automatically learns normal operating conditions, compresses sensor data into meaningful feature representations, reconstructs the input, and identifies defective products using reconstruction error. This approach enables accurate, real-time quality control, reduces manual inspection, and improves overall production efficiency.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand